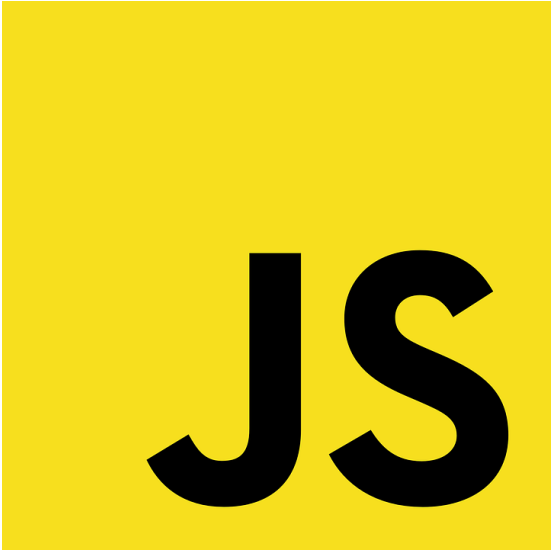
Chap 3 - JavaScript



Plan du Cours

1. Introduction
2. Variables, fonction, structure de contrôle
3. Les objets JavaScript
4. DOM
5. Evènements
6. Dessiner en JavaScript
7. Conclusion et références

Outline

1. Introduction
2. Variables, fonction, structure de contrôle
3. Les objets JavaScript
4. DOM
5. Evènements
6. Dessiner en JavaScript
7. Conclusion et références

Contexte d'utilisation

- JavaScript(JS) s'utilise dans le contexte du Web principalement au travers d'un navigateur Internet .
- JavaScript permet l'exécution de code informatique intégré à des pages Web .
- JavaScript est un des outils permettant de développer du "Web Dynamique".
- Réduit l'échange de données avec le serveur.

Créé en 1995 par Brendan Eich (pour la Netscape Communication Corporation):

- La version ES5 date de 2009.
- La version ES6 date de 2016.

Principales caractéristiques

- JavaScript est interprété, faiblement typé et sans notion de pointeurs.

```
1  console.log( "20" + 46) ; // affiche "2046"
2  let ma_var = 2046;
3  ma_var = 'ITMFL'; // pas d'erreur de typage
```

- Types de base simples (booléen, nombre) et chaîne, tableau.
- Les fonctions et les variables peuvent être déclarées n'importe où .
- Les fonctions admettent un nombre de paramètres variable .
- Javascript est indépendant du matériel et donc portable.
- Langage basé sur les objets mais simplifié et informel.

JavaScript - Pour quoi faire?

- Valider des champs de formulaires.
- Enregistrement et lecture de cookies.
- Modification dynamique de l'aspect de certains éléments de la page.
- Affichage et masquage des éléments.
- Déplacement d'éléments sur la page.
- Capture de certains événements et modification de la page en conséquence.
- Défilement de contenu.

Ou se trouve un script JavaScript?

A l'intérieur de la balise `<script>`

```
1  <html lang="fr">
2
3  <body>
4  <! -- Mauvaise pratique ! -- >
5  <script>
6  let nom='Sciamma';
7  let prenom='Celine';
8  let naissance = 1978;
9  let annee_courante = new Date().getFullYear();
10 let phrase=`La realisatrice ${prenom + ' '+ nom } a cette
11 annee ${annee_courante - naissance} ans`;
12   document.write( phrase );
13 </script>
14 </body >
15 </html >
```

Ou se trouve un script JavaScript?

A l'intérieur de la balise `<script>`

```
1  <html>
2  <head>
3  <!-- Mauvaise pratique ! -->
4  <script>
5  function decubique () {
6  return Math.ceil(6* Math.random() ) ;
7  }
8  </script>
9  </head>
10 <body>
11 <h1> Tirage d un de a 6 faces </h1>
12 <script>
13 document.write('<p> Tirage=' + decubique () + '</p>') ;
14 </script>
15 </body>
16 </html>
```

Chargement de programmes JavaScript en HTML

- balise `<script>`
- à placer dans la partie `<head>`
- ou (très fortement recommandé) : en fin de `<body>`:
  - permet de ne pas ralentir l'affichage de la page avec le téléchargement du JavaScript
  - permet de s'assurer que la page est entièrement chargée lorsque le code JavaScript va accéder à des éléments de la page

```
1  <html>
2    <head>
3    ...
4    </head>
5    <body>
6    ....
7    <script src="monscript.js" > </script>
8    </body>
9  </html>
```

Ou placer les balises `<script>`

- Entre `<head>` et `</head>` :
  - Code chargé en même temps que le reste de la page
  - Non exécuté (attention les affectations de variables non effectuées).
- Entre `<body>` et `</body>`
  - Code chargé en même temps que le reste de la page.
  - Si exécutable : exécuté en même temps que les autres balises HTML.
  - Si définition de fonctions : fonctions stockées et non exécutées Attention au temps de chargement !!

Versions de JavaScript

- Deux versions principales cohabitent : ES5 et ES6
- ES6 ajoute des constructions à ES5 mais ne les remplace pas
- Le passage de ES5 à ES6 se fait progressivement au fil des versions des navigateurs
- Presque tout ES6 est implanté dans les navigateurs récents, mais pas les anciens

Mode strict

- JavaScript possède 2 modes de fonctionnement
  - le mode normal ou sloppy
  - le mode strict
- Le mode strict peut être activé :
  - pour un fichier complet avec en début le tag suivant :

```
1  ' use strict '
```

  - par fonction

```
1  function fo nctionEnModeStrict () {
2    'use strict';
3    ...
4  }
```

- Le mode strict génère plus de warnings et d'erreurs mais est souvent plus rapide.

Outline

1. Introduction
2. Variables, fonction, structure de contrôle
3. Les objets JavaScript
4. DOM
5. Evènements
6. Dessiner en JavaScript
7. Conclusion et références

Les variables

- Pas d'initialisation par défaut (sauf par la valeur undefined).
- Déclaration : Mot clé `let` (ES6).
- Le type est dynamique:

```
1  var x=5; // int
2  x=5.3; // float
3  x="une chaine de caractere";
4  x='une chaine de caractere';
```

- Recommandation : éviter les changements de type pour une variable.
- Le nom de variable est formé de `lettre, chiffre, $` ou `_`.

Déclaration et portée des variables

Deux syntaxes de déclarations de variables

- Mot-clé var (ES5) : la portée (scope) est l'ensemble de la fonction où elle est déclarée.
- Mot-clé let (ES6) : la portée (scope) est limitée au bloc où elle est déclarée
- Une variable déclarée en dehors de tout bloc est globale

```
1  function f () {
2    var v = 2046;
3    if ( v < 3000) {
4      let w = 3000 -2046;
5      var z = w + v ;
6      document.writeln(v , w ) ; // OK
7    }
8     document.writeln(v ,z , w ) ; // pas OK
9  }
```

Variable globale

- Une variable *declarée* sans le mot clef `var` ou `let` est une variable globale.

```
1  <script>
2  var prenom="Jean"; //var globale
3  function paul()
4  {
5    var prenom = "Paul" ; // var locale a paul()
6    document.writeln("Dans la fonction paul = " + prenom);
7  }
8  document.writeln("Avant la fonction paul = "+ prenom);
9  paul();
10 document.writeln("Apres la fonction paul = "+ prenom);
11 </script>
```

popup alert et prompt

- La fonction prompt sert à ouvrir une fenêtre avec un champs d'entrée pour récupérer une valeur de l'utilisateur.
- La fonction alert sert à ouvrir une fenêtre popup qui contient un message.

```
1 <script>
2  var prenom= prompt('Donnez votre prenom:');
3  alert('Bonjour' + prenom);
4 </script>
```

Les operateurs

Les opérateurs arithmétiques sont similaires aux opérateurs en C.

Signe	Signification	Exemple
=	a la valeur affectation	x=5
+	addition	x + 3
-	soustraction	x - 3
*	multiplication	x*2
/	division	x /2
%	modulo reste de la division	x%5

Il y a aussi les opérateurs associatifs `+=, -=, *=, /=`
Et les opérateurs d'incrémentation `++,--`

Fonctions

- Les fonctions sont similaires au C.
- Seule différence : il n'y a pas de typage!

```
1  function nom_fonction(arguments)
2  {
3     //...  instructions ...
4     return(valeur_renvoyee);
5  }
```

- Exemple : fonction d'élévation au carré

```
1  function carre(x)
2  {
3     return(x*x);
4  }
```

- Depuis ES6 : nouvelle notation des fonctions avec =>:

```
1  let produit = (a , b ) = > { return a * b ;}
2  let wkw = () => { return 2046;}
3  let incrBy1 = ( x ) => x +1;
4  document.writeln( wkw ) ; // affiche  "function wkw () {...}"
5  document.writeln( wkw () ) ; // affiche 2046
6  document.writeln( wkw.name ); // affiche "wkw"
7  document.writeln( produit(31 ,66) ) ; // affiche "2046"
```

Structures de contrôles

Les structures de controles sont les mêmes que dans le langage C.

- Structure if..else

```
1  if (condition) { instructions;} else { instructions;}
```

- Structure for

```
1  for (i = 1;i <= 10;i++) { instructions; }
```

- Structure while

```
1  while(condition) {instructions;}
```

- Aussi switch

```
1  switch(condition) { case ..: ... default: ... }
```

Opérateurs de comparaison, opérateurs logiques

Il y a huit opérateurs de comparaison

Symbole	Nom	Exemple	Résultat
$===$	égal à (contenu et type)	$x===11$	true
$==$	égal (avec transtypage)	$x==11$	true
$<$	inférieur	$x<11$	false
$<=$	inférieur ou égal	$x<=11$	true
$>$	supérieur	$x>11$	false
$>=$	supérieur ou égal	$x>=11$	true
$!=$	différent (avec transtypage)	$x! =11$	false
$!==$	différent (contenu ou type)	$x! =11$	false

Voici les trois opérateurs logiques:

Symbole	Nom	Exemple	Résultat
&&	et	(cond1) && (cond2)	cond1 et cond2
||	ou	(cond1) || (cond2)	cond1 ou cond2
!	not	!cond	not cond

Exemple de if : tranche d'age

Code qui demande l'age et donne la tranche d'age de l'utilisateur.

```
1  <script>
2  var age = parseInt(prompt('Quel est votre age ?'));
3  if (1 <= age && age <= 6) {
4    alert('Vous etes un jeune enfant.');
5  }
6  else if (7 <= age && age <= 11) {
7    alert ('Vous etes un enfant qui a atteint l\'age de raison.');
8  }
9  else if (12 <= age && age <= 17) {
10  alert ('Vous etes un adolescent.');
11 }
12 else if (18 <= age && age <= 120) {
13   alert ('Vous etes un adulte.');
14 } else {
15   alert ('Erreur !!');
16 }
17 </script>
```

Outline

1. Introduction
2. Variables, fonction, structure de contrôle
3. Les objets JavaScript
4. DOM
5. Evènements
6. Dessiner en JavaScript
7. Conclusion et références

Les Objets Javascript

Quelques notions de programmation objet.

- Un objet est en quelque sorte une version enrichie des structures en C
- Un objet c'est
  - Un ensemble de propriétés (i.e. de valeurs)
  - Un ensemble de méthodes (i.e. de fonctions).
- Syntaxe

```
1 //Creation
2 var refobjet= new Objet();
3 //Acces aux proprietes
4 refobjet.nom_propriete=valeur;
5 //Appel de methode
6 refbjet.nom_methode(liste_arguments);
```

Les objets en JavaScript

- Les objets natifs: Math, String, Array, Date, ...
- Les objets des pages web: document, navigator, window,
- Les objets créés dans le script.

L'objet Date

- Initialisation `var dtNow = new Date();`
- en miliseconde : on rentre la date en miliseconde.

```
1  var dtMilliseconds = new Date(3586253629);
```

- Avec une chaine de caractère (les heures,minutes,secondes sont facultatives)

```
1  var dtchaine=new Date("March 12, 1980 12:20:25");
```

- Elle peut être spécifier sous forme numérique (annee,mois,jour,heure,minute,seconde et miliseconde)

```
1  var dtchaine=new Date(1980,11,24,11,35,51,0);
```

- On peut récupérer une partie de l'information grace aux méthodes `getFullYer,GetHours,GetMillisecons,...)`
- On peut convertir une date en chaine de caractère avec la méthode `toString`

L'objet Math

- Des propriétés:

```
1  Math.E ; // le nombre d'Euler
2  Math.PI ;
3  Math.LN10 // le logarithme neperien de 10
4  Math.LN2 //  le logarithme neperien de 2
```

- Des méthodes:
  - lfoor : calcule le plus grand entier inférieur ou égal
  - round : arrondit à l'entier le plus proche
  - min : calcule le minimum d'un ensemble de nombres
  - max : calcule le maximum d'un ensemble de nombres
  - random : renvoie un nombre pseudo aléatoire dans $[0, 1]$
  - en aussi: abs,ceil,cos,sin,exp,sart,log,pow,...

Plus d'info sur l'API Math de JS:
`https://www.w3schools.com/js/js_math.asp`.

L'objet String

- L'objet string permet de définir, modifier, manipuler des chaines de caractère.
- Déclaration

```
1  var prenom ="Jean";
2  //ou
3  var prenom = new String("Jean");
4  //ou
5  var prenom = new String();
6  nom.valueOf="Jean";
```

Principales proprétés/méthodes de l'objet String

Méthode	Description	Paramètre
`valueOf`	valeur contenue	
`length`	longueur de la chaine	
`concat`	Concatène deux chaines	chaine à ajouter
`split`	Découpe en fonction d'un séparateur	le séparateur et nombre maximal
`slice`	section d'une chaîne	début et fin de la section
`substring`, `substr`	Retourne une sous-chaîne	début et fin de la sous-chaine
`match`,`replace` `search`	recherche,remplace avec une expression régulière	la regex
`toLowerCase` `toUpperCase`	Changement de casse	aucun

Exemple

```
1  nom = new String("Martin");
2  // utilisation de la propriete length
3  alert("longueur du nom =" + nom.length);
4  // utilisation  de la methode substring
5  alert("caractere 2 et 3 =" + nom.substring(1,3));
```

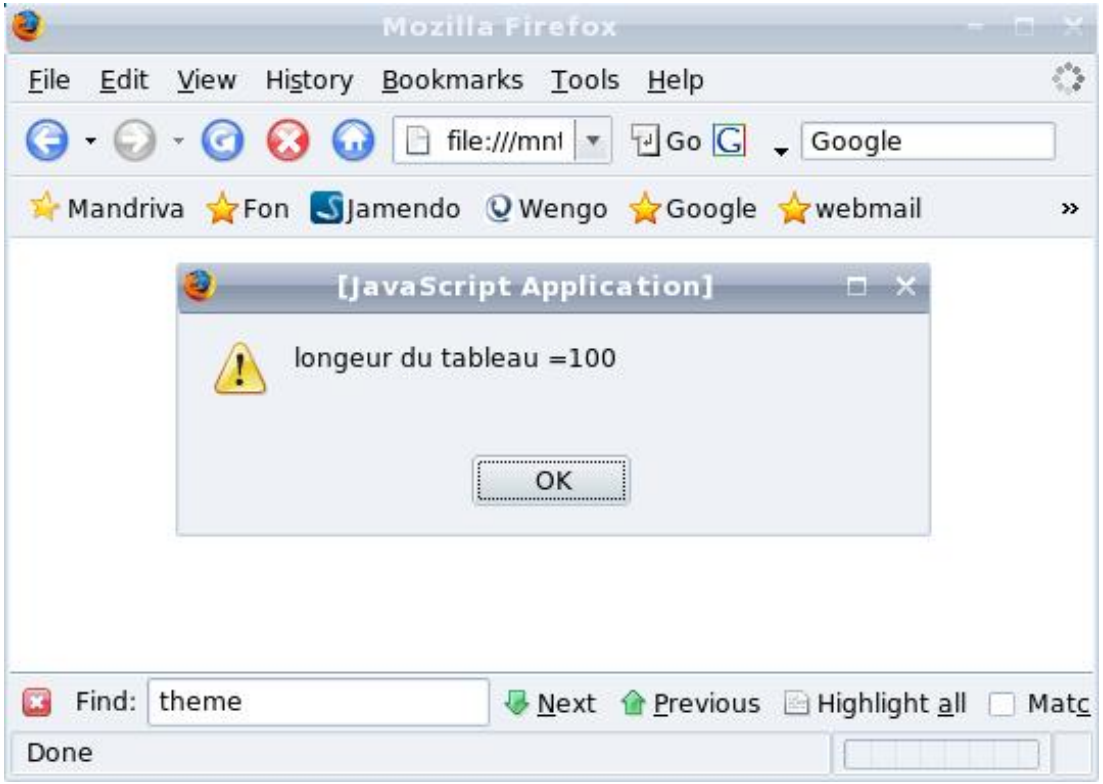
Les tableaux - l'objet Array

- Contrairement aux langages traditionnels informatiques, il suffit de déclarer que c'est une table, (array) en anglais, sans indiquer de taille particulière.
- Les éléments sont repérés par un indice, la numérotation commence à 0.
- Déclaration

```
1  // declaration avec l'objet Array
2  var MeubleClassique = new  Array();
3  // On peut stocker
4  MeubleClasssique[0]="Armoire Louis 15";
5  MeubleClasssique[1]="Fauteuil Second Empire";
6  // On aurait pu le remplir directement
7  var MeubleClassique = new  Array("Armoire Louis 15","Fauteuil
8  Second Empire");
9  // Ou ce qui est equivalent
10 Personne = ["Jean","Annie","Marcel"];
```

On peut connaitre la longueur du tableau à l'aide de la propriété `length`

```
1  var MeubleClassique = new  Array();
2  // On peut stoquer
3  MeubleClassique[0]="Armoire Louis 15";
4  MeubleClassique[99]="Fauteuil Second Empire";
5  alert("longeur du tableau =" + MeubleClassique.length);
```



Les opérations push,pop, shift

- push(element) ajoute un ou des éléments à la fin,

```
1  var myArray = ['Rafael', 'Mathilde'];
2  myArray.push('Ahmed'); // Ajoute  Ahmed a la fin du tableau
3  myArray.push('Jerome', 'Guillaume'); // Ajoute Jerome et
       Guillaume a la fin
```

- pop() supprime le dernier élément
- shift() supprime l'éléments de tête
- unshift(element) ajoute un élément en tête de tableau

```
1  var myArray = ['Rafael', 'Mathilde', 'Ahmed', 'Jerome', 'Guillaume'
       ];
2  myArray.shift(); // Retire  Rafael
3  myArray.pop(); // Retire  Guillaume
4  alert(myArray); // Affiche  Mathilde,Ahmed,Jerome
```

Tableau et String : split et join

- split découpe une chaine tableau (le paramêtre est le séparateur):

```
1  var cousinsString = 'Jerome Guillaume Paul',
2  cousinsArray = cousinsString.split(' '); // Avec les espaces, on
       a trois items
3  alert(cousinsString);
4  alert(cousinsArray);
```

- join fabrique une chaîne de caractères des éléments du tableau:

```
1  var cousinsString_2 = cousinsArray.join('-');
2  alert(cousinsString_2);
```

Les tableaux associatifs

- On accède aux élément du tableau par un identifiant.
- On a (évidement) les même méthode que pour le tableau indicé (même objet)
- Le parcours d'un tableau associatif se fait grace à `for x in TabAss`.

```
1   var family = {
2  self: 'Rafael',
3  sister: 'Mathilde',
4  brother: 'Ahmed',
5  cousin_1: 'Jerome',
6  cousin_2: 'Guillaume'
7  };
8  var id = 'sister';
9  alert(family[id]); // Affiche :  Mathilde
10 for (var id in family) {
11   writeln('family['+id+']='+family[id]+'<br>');
12 }
```

L'objet Math

Cet objet contient différente constantes mathématiques

Math.E	=	valeur de e
Math.LN10	=	le logarithme naturel de 10
Math.LN2	=	logarithme naturel de 2
Math.PI	=	valeur de π

- Des méthodes mathématiques

  `Math.sin`,`Math.cos`, `Math.tan`,`Math.acos`,`Math.asin`,`Math.atan`

- Une méthode pour générer des nombres aléatoires `Math.random`.

Créer ses propres objets : en ES5

```
1  // definition de l'objet (proprietes et methodes)
2  var velo = {
3   marque : "Giant",
4  bequille : true,
5  couleur : "bleu",
6  vitesse : 0,
7  demarrer : function() { this.bequille = false ;} ,
8  accelerer : function( x ) { if (! this . bequille )
9      this . vitesse += x ; } ,
10 repeindre : function( c ) { this.couleur = c ;}
11 };
12 // on appelle les methodes
13 velo.demarrer() ;
14 document.writeln( velo . bequille ) ; // affiche " false "
15 velo.accelerer(20) ;
16 document.writeln( velo . vitesse ) ; // affiche "20"
```

Créer ses propres objets : en ES6

Légère modification de la syntaxe pour les méthodes:

```
1  var velo = {
2   marque : "Giant",
3  bequille : true,
4  couleur : "bleu",
5  vitesse : 0,
6  demarrer() { this.bequille = false ;} ,
7  accelerer( x ) { if (! this . bequille )
8      this . vitesse += x ; } ,
9  repeindre( c ) { this . couleur = c ;}
10 };
11 // on appelle les methodes
12 velo.demarrer() ;
13 document.writeln( velo . bequille ) ; // affiche " false "
14 velo.accelerer(20) ;
15 document.writeln( velo . vitesse ) ; // affiche "20"
```

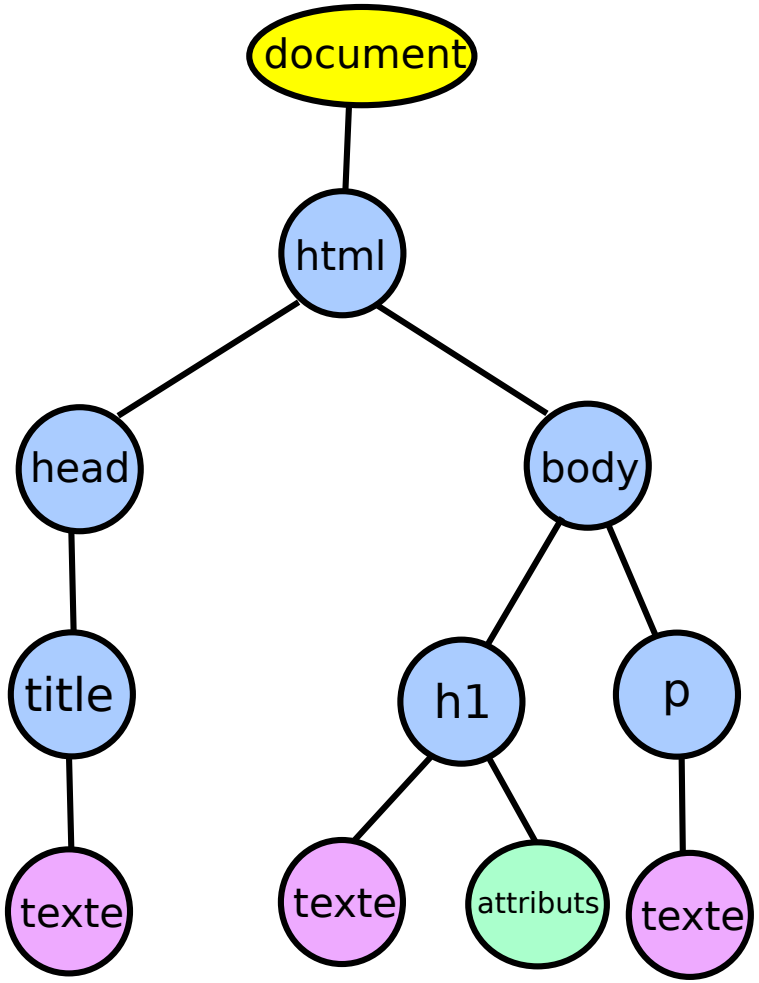
Outline

1. Introduction
2. Variables, fonction, structure de contrôle
3. Les objets JavaScript
4. DOM
5. Evènements
6. Dessiner en JavaScript
7. Conclusion et références

DOM : rappel

- Les navigateurs disposent d'un interprète JavaScript embarqué.
- JavaScript interagit avec la page web via le DOM
- Réagit à des évènements

```
1  <!DOCTYPE html>
2  <html lang="fr" >
3   <head>
4     <title> Page pour un DOM </
        title>
5   </head>
6   <body>
7     <h1 id="titre"
8    class = "bleu"> Bienvenue </h1>
9    <p>A l'Universite de Perpignan
       </p>
10   </body>
11 </html>
```



L'objet Window

C'est l'objet de base qui contient tous les autres objets.

- Création : la méthode open

```
1  var newWindow = window.open(url,titre,longueur,largeur,toolbar,
       location)
```

La fonction open renvoie une référence `newWindow`. Cette référence permet d'agir sur la fenêtre ouverte.

- Contrôle de la taille : les méthodes `resizeBy` et `reseizeTo`

```
1  // Ajoute lx (resp. ly) a la largeur (resp. longueur)
2  newWindow.resizeBy(lx,ly)
3  // Fixe la largeur (resp. longueur) a  lx (resp. ly)
4  newWindow.reseizeTo(lx,ly)
```

- Déplacement : la méthode `moveTo(x,y)` place le coin haut-gauche de la fenêtre au point de l'écran de coordonnée (x, y).

L'objet Document

- L'objet document représente la page HTML en elle même.
- Il contient tous les éléments de la page html vu comme des objets.

```
1  // la 2 eme image
2  document.images[2]
3  // le lien identifie par id="montagne"
4  // ou name (pour les elements du formulaire)
5  document.links["peace"]
6  // on recupere l'element de texte de la balise <a> "peace"
7  let text = link.textContent;
```

- Pour chaque objet, les attributs associé à la balise sont des attributs de l'objet

```
1  document.images[2].src // source de l'image
```

Fonction d'accès aux éléments d'un document

Il existe trois fonctions pour accéder aux éléments d'une page web:

- getElementById("monid"): retourne le noeud dont l'identifiant est "monid"
- getElementsByTagName("mabalise") : retourne le noeuds qui sont des balise "mabalise"
- getElementsByClassName("maclasse") : retourne les noeuds dont la classe est maclasse

La méthode getElementById()

- Cette méthode permet de récupérer la référence à un élément (balise) ayant un id.

```
ref_element = getElementById(idelement)
```

- On peut modifier le style de l'élément capturé

```
ref_element.style.propriete=new_value;
```

- Pour modifier les autres attibuts : meme principe

```
ref_element.attribut=new_value;
```

Remarque : les propriétés ne s'écrivent pas tout à fait pareil qu'en CSS se référer à:

`https://www.w3schools.com/jsref/dom_obj_style.asp`

Exemple : changement de couleur d'un div

Un div bleu dans le HTML, que l'on change en vert dans le JavaScript:

```
1  <div id="div" style="position:absolute;
2                       width:100px; height:100px;
3                       background-color:blue;" >
4  </div>
5  <script>
6  var div=document.getElementById("mydiv");
7  div.style.backgroundColor="green";
8  </script>
```

Sélectionner des éléments du DOM : exemple

Un code HTML

```
1 <html>
2 <head> ... </head>
3 <body>
4 <section class=" main">
5 <p id="p1"> paragraphe 1 </p>
6 <p id="p2"> paragraphe 2 </p>
7 <p id="p3"> paragraphe 3 </p>
8 </section>
9 </body> </html>
```

Un script dans la page

```
1 let premier = document.getElementById("p1") ;
2 document.writeln( premier.nodeName, premier.id) ; // affiche " P p1
     "
3
4 let paragraphes = document.getElementsByTagName("p") ;
5 document.writeln( paragraphes[1].nodeName , paragraphes[1].id ) ; //
      affiche "P p2 "
6
7 let mains = document.getElementsByClassName("main");
8 document.writeln( mains[0].nodeName , mains[0].className ) ; //
     affiche "section main "
```

Modifier les attributs

Les fonctions getAttribute() et setAttribute() permettent de modifier un attribut d'un élément de la page.

```
1  <a id="myLink" href="http://www.un_lien_quelconque.com">Un lien
       modifie
2  dynamiquement</a>
3  <script>
4  var link=document.getElementById('myLink');
5  var href=link.getAttribute('href'); // On recupere l'attribut  href
6  alert(href);
7  link.setAttribute('href', 'http://blog.crdp-versailles.fr/rimbaud/')
       ; // on edite
8  </script>
```

Créer et insérer des éléments (1/2)

- parentNode permet d'accéder à l'élément parent d'un élément.
- nodeType et nodeName permettent de vérifier le type et le nom d'un noeud.
- firstChild et lastChild permettent d'accéder au premier et au dernier élément enfant d'un noeud.
- childNodes retourne un tableau contenant la liste des enfants d'un élément.

Créer et insérer des éléments (2/2)

L'ajout d'un élément se fait en trois temps : on crée l'élément, on lui affecte des attributs, on l'insère dans le document.

```
1  <div><p id="myP">Un peu de texte <a>et un lien</a></p></div>
2  <script>
3  var newLink = document.createElement('a');
4  newLink.id = 'sdz_link';
5  newLink.href = 'http://blog.crdp-versailles.fr/rimbaud/';
6  newLink.title = 'Decouvrez le blog de la Classe Actu !';
7  newLink.setAttribute('tabindex', '10');
8  // le nouvel element est le dernier enfant du paragraphe d'id 'myP
       '
9  document.getElementById('myP').appendChild(newLink);
10 var newLinkText = document.createTextNode("Le Tonnerre de Rimbaud"
       );
11 newLink.appendChild(newLinkText); // ces deux lignes pour ajouter
       le texte
12 </script>
```

Outline

1. Introduction
2. Variables, fonction, structure de contrôle
3. Les objets JavaScript
4. DOM
5. Evènements
6. Dessiner en JavaScript
7. Conclusion et références

Les évènements

- Les évènements sont des actions de l'utilisateur
- Les évènements liés à la souris
  - Clics souris.
  - Mouvement de la souris.
  - Placement de la souris.
  - Redimensionnement.
- Tapes au clavier (différentes touches).
- Changement de Focus (zone active).

Quelques évènements du DOM en JavaScript

Évènement	Type de noeud	Action détectée
load	document,frameset	Chargement complet du DOM
click	element	Clic de souris
mousedown	element	Bouton de souris enfoncé
mouseup	element	Bouton de souris enfoncé
mouseup	element	Bouton de souris relâché
mouseover	element	Passage du pointeur au dessus
mouseout	element	Sortie du pointeur
keypress	element	Touche de clavier pressée
blur	element	Perte de focus
focus	element	acquisition du focus
change	element	modification de la valeur
resize	document, element	changement de la taille
scroll	document, element	utilisation de la molette

Méthode 1 (à éviter): évènement onclick dans l'attribut

Évènement classique en informatique, le clic de la souris. Ci-dessous un exemple où une fenêtre `alert` s'ouvre lorsque l'on clique sur un bouton

```
1  ...
2  <script>
3  function bouton()
4  {
5    alert("Je vous avais dit de pas cliquer !");
6  }
7  </script>
8  ....
9  <form>
10 <input type="button" value="Ne cliquez Pas!"
11 onclick="bouton()">
12 </form>
```

Mauvaise pratique

Cette pratique est mauvaise et ne devrait pas être utilisée.

Méthode 2 (à éviter): propriété on<event> des noeuds

- Donner une valeur à la propriété on<event> d'un nœud du DOM
  - load ⇒ onload
  - click ⇒ onclick
  - etc.
- Lorsque l'évènement se produit, cette valeur est utilisée comme gestionnaire d'évènement

```
1  node.onclick = () = > { ... };
2  node.onmouseover = animateButton ;
3  window.onload = () = > { ... };
```

Un seul gestionnaire

Cette méthode ne permet pas d'enregistrer plusieurs gestionnaires (handler) pour un même évènement.

Méthode 2 (à éviter): exemple

```
1  <!DOCTYPE html>
2  <html>
3    <head>
4      <title> Gestionnaire d evenement - Methode 2 </title>
5    </head>
6    <body>
7      <p id="par1"> Paragraphe 1 (onclick) </p>
8      <p id="par2"> Paragraphe 2 (onmouseover) </p>
9      <script>
10       let par1=document.getElementById("par1");
11       let par2= document.getElementById("par2");
12       par1.onclick = ( event ) => { alert ("Click : paragraphe 1")
             ;};
13       par2.onmouseover = ( event ) => { alert ("Mouseover :
             paragraphe 2");};
14     </script>
15   </body>
16 </html>
```

Méthode 3 (bien) : auditeur

- Enregistrement d'un auditeur (ou listener) sur un noeud pour un évènement:

```
1  node.addEventListener( "evt", handler ) ;
```

  - evt : l'évènement concerné.
  - handler : une fonction revevant un objet event en paramètre.

- On peut enregistrer plusieurs auditeurs sur le même noeud pour un même évènementment. Exemple pour un lien:

```
1  lien.addEventListener("click", (event) => { alert("clic!"); });
2  lien.addEventListener("click", (event) => { document.writeln("
       clic!"); });
```

Méthode 3 (bien): exemple

```
1  <!DOCTYPE html>
2  <html>
3    <head>
4      <title> Gestionnaire d evenement - Methode 2 </title>
5    </head>
6    <body>
7      <p id="par1"> Paragraphe 1 (onclick) </p>
8      <p id="par2"> Paragraphe 2 (onmouseover) </p>
9      <script>
10       let par1=document.getElementById("par1");
11       let par2= document.getElementById("par2");
12       par1.addEventListener("click", ( event ) => { alert("Click :
             paragraphe 1");});
13       par1.addEventListener("click", ( event ) => { document.writeln
             ("Onclick : paragraphe 1");});
14       par2.addEventListener("mouseover", ( event ) => { document.
             writeln("Mouseover : paragraphe 2");});
15     </script>
16   </body>
17 </html>
```

Méthode 3: complément

DOM chargé

Il est important que le DOM soit entièrement chargé avant de sélectionner des noeuds et d'ajouter des listeners.

```
1  'use strict'
2  window . addEventListener ( "load" , () = > {
3   // on recupere des elements
4   let elt=document.getElementById(... ) ;
5   ...
6   // on a joute des auditeurs
7   elt.addEventListener( ... );
8   ...
9  });
```

Propriété et méthodes des évènements

- Un auditeur reçoit comme seul argument un objet event.
- Selon le type de l'évènement, on a accès à différentes propriétés/méthodes:
  - Pour accèder à la cible: event.target ou this dans une function.
  - Pour annuler l'évènement envent.preventDefault().

Outline

1. Introduction
2. Variables, fonction, structure de contrôle
3. Les objets JavaScript
4. DOM
5. Evènements
6. Dessiner en JavaScript
7. Conclusion et références

La balise HTML canvas

- L'élément canvas se comporte comme un e image:

```
1  <canvas id="canvas" width="300" height="300">
2   Le navigateur ne prend pas en charge le canevas.
3  </canvas>
```

- Sur un objet associé à canvas:
  - La méthode getContext donne accès au contexte graphique du canevas et permet ensuite de dessiner.
  - Zone de dessin : origine $(0, 0)$ en haut à gauche du canevas.

Dessiner un rectangle

- fillRect: dessine un rectangle "plein"

```
1   fillRect(x,y,largeur,hauteur);
```

- strockRect: dessine un contour rectangulaire"

```
1   strockeRect(x,y,largeur,hauteur);
```

- clearRect: efface une zone rectangulaire

```
1     clearRect(x,y,largeur,hauteur);
```

Tracer des trajets, lignes

Cela se fait en quatre étapes:

- Création du trajet avec beginPath()
- Séquence d'instructions pour dessiner le trajet:
  - moveTo(x,y): déplacement en (x,y) sans trace.
  - lineTo(x,y): trace une ligne depuis la position courante jusqu'à (x,y).
- On ferme le trajet avec closePath().
- On le trace avec fill() ou on le remplit avec stroke().

Tracer des arcs, cercles ou disques

- Création du trajet avec beginPath()
- arc(x,y,rayon,angleinitial,anglefinal,antihoraire) dessine un arc de cercle.
- Cercle: utiliser arc avec angleinitial=0 et anglefinal=2*Math.PI.
- On ferme le trajet avec closePath().
- On le trace avec fill() ou on le remplit avec stroke().

```
1  const canvas=document.getElementById( 'canevas');
2       const contexte = canvas.getContext( '2d');
3       contexte.fillStyle="orange";
4       contexte.beginPath();
5       contexte.arc(100,100,100,Math.PI/2,2*Math.PI,false);
6       contexte.closePath();
7       contexte.fill();
```

Les couleurs

Deux propriétés importantes du canevas:

- fillStyle(couleur): définit la couleur utilisée pour remplir.
- strokeStyle(couleur): définit la couleur utilisée lors d'un tracé.
- couleur est une chaine représentant une couleur CSS.

Exemple :

```
1  contexte . fillStyle = " orange " ;
2  contexte . fillStyle = " # FFA500 " ;
3  contexte . fillStyle = " rgb (255 ,165 ,0) " ;
4  contexte . fillStyle = " rgba (255 ,165 ,0 ,1) " ;
```

Outline

1. Introduction
2. Variables, fonction, structure de contrôle
3. Les objets JavaScript
4. DOM
5. Evènements
6. Dessiner en JavaScript
7. Conclusion et références

Synthèse : ce que vous devez savoir faire

Savoir faire des programmes usuels en JavaScript

- Manipuler les variables de types numérique, chaîne de caractères et tableau.
- Il faut savoir définir des fonctions.
- Il faut aussi savoir définir des objets.

Il faut savoir manipuler le DOM

- Accéder et modifier des attributs ou propriété de style.
- Ajouter ou supprimer des éléments.

Il faut savoir gérer les évènement

- Ajouter des auditeurs aux event.

Il faut savoir dessiner des formes de base (rectangle, cercle, arc, tracé).

Quelques références

- Le cours Programmation Web de Etienne André - Université Paris Nord.
- Livre de Johann Pardanaud et Sebastien de la Marck. “Decouvrez le langage JAVASCRIPT”, ed. Eyrolles.
- Le site `https://www.w3schools.com/js/default.asp`.